Task 5

Deep Learning and Predictive Analytics Techniques

Predictive Analytics & AI Model Deployment - Customer Churn Prediction

Objective. Implement a neural network and predictive-analytics techniques (importance, lift, segmentation, ROI).

Part A - Deep learning (neural network)

TensorFlow and PyTorch could not be installed offline, so the deep-learning component uses scikit-learn's **MLPClassifier** - a fully-connected feed-forward neural network trained with back-propagation. It runs inside the same preprocessing pipeline as every other model.

Network design

- Input: 44 preprocessed features; hidden layers ReLU; output: sigmoid probability of churn.
- Optimiser Adam (lr 0.001), mini-batches of 256, L2 penalty 0.001, log-loss.
- **Early stopping** on a 10% validation split (patience 10) to prevent overfitting: stopped after 16 epochs.
- Three architectures were compared on a hold-out split; **128-64** was selected and re-fitted on the full training set (14,209 trainable parameters).

Architecture (hidden layers)	Validation ROC-AUC
64	0.7991
128-64	0.7995
128-64-32	0.789

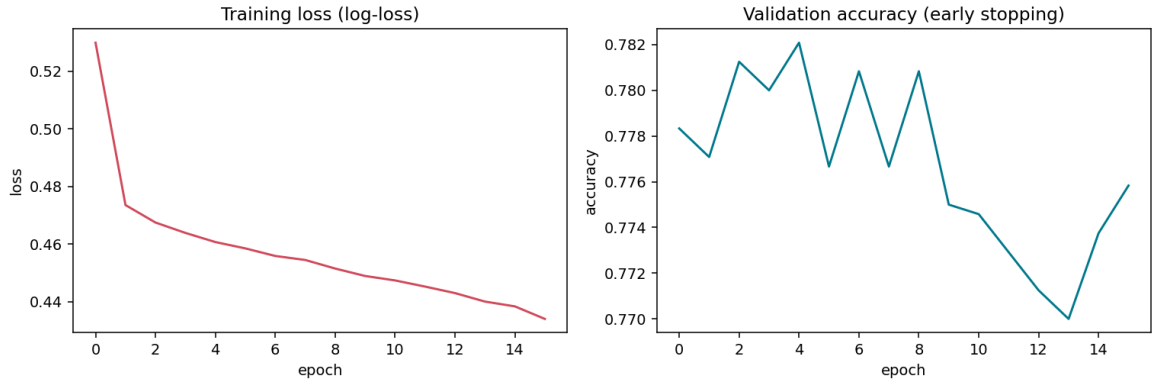


Figure 1 - Training loss and validation accuracy per epoch

On the test set the MLP reaches ROC-AUC 0.786 - on par with boosting but not better than logistic regression (Task 4). With ~24,000 tabular rows and a mostly additive signal, deep networks have little to gain; they would matter more with far larger data, text or image inputs.

Part B - Predictive analytics on the selected model (Logistic Regression)

1. What drives churn? (permutation importance)

Each raw input column is shuffled in turn on a 3,000-row test sample and the resulting drop in ROC-AUC is measured. Tenure, total charges (which encodes tenure x price) and contract type dominate, followed by satisfaction and payment behaviour.

Rank	Feature	AUC drop when shuffled
1	tenure_months	0.1887 (+/- 0.0084)
2	total_charges	0.0932 (+/- 0.0058)
3	contract_type	0.0890 (+/- 0.0052)
4	satisfaction_score	0.0380 (+/- 0.0033)
5	late_payments	0.0145 (+/- 0.0017)
6	monthly_charges	0.0121 (+/- 0.0016)
7	num_support_tickets	0.0113 (+/- 0.0022)
8	last_login_days	0.0083 (+/- 0.0005)
9	num_products	0.0052 (+/- 0.0024)
10	discount_offer	0.0039 (+/- 0.0010)
11	age	0.0022 (+/- 0.0005)
12	payment_method	0.0020 (+/- 0.0010)

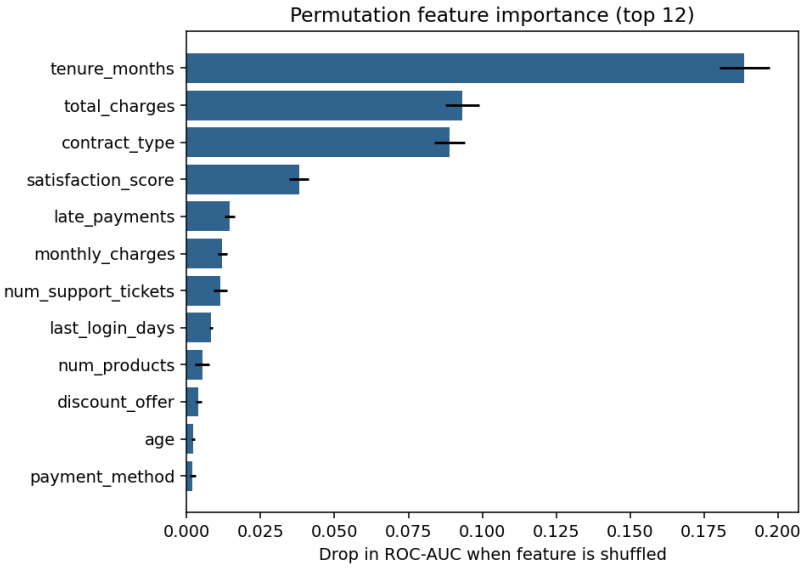


Figure 2 - Permutation importance

2. How well does the model rank customers? (lift and gains)

Sorting test customers by predicted risk, the top decile churns 2.58x more than average, and contacting the top 20% of customers reaches 44.5% of all churners.

Decile	Customers	Churn rate	Lift	Cum. % of churners captured
1	600	69.5%	2.58	25.8%
2	600	50.3%	1.87	44.5%
3	600	42.2%	1.57	60.2%
4	600	30.7%	1.14	71.5%
5	600	24.5%	0.91	80.6%
6	600	20.8%	0.77	88.4%
7	600	14.5%	0.54	93.8%

Decile	Customers	Churn rate	Lift	Cum. % of churners captured
8	600	8.0%	0.30	96.7%
9	600	7.3%	0.27	99.4%
10	600	1.5%	0.06	100.0%

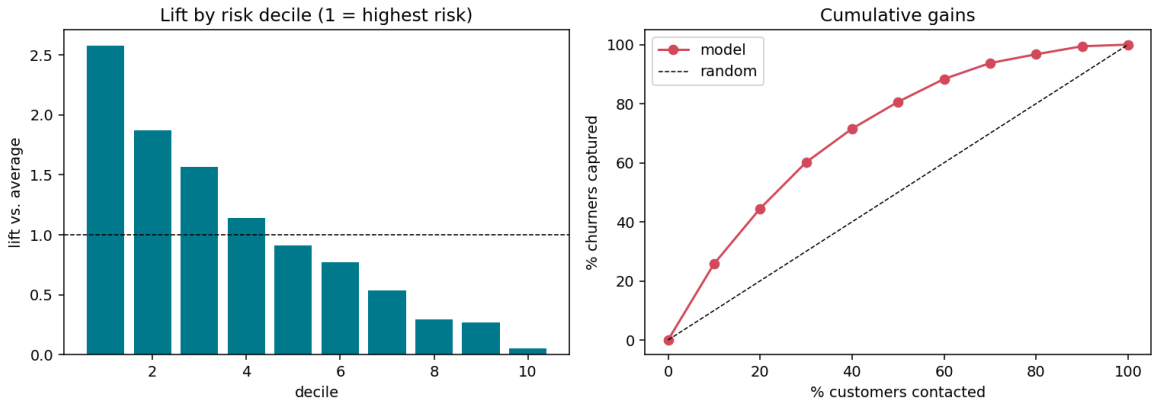


Figure 3 - Lift by decile and cumulative gains

3. Risk segmentation

Predicted probabilities are grouped into Low (<25%), Medium (25-55%) and High (≥55%). Predicted and actual churn rates agree closely in each segment, confirming the probabilities are well calibrated and can be used directly for targeting.

Risk segment	Customers	Share	Avg predicted	Actual churn rate
Low	3321	55.4%	11.3%	11.5%
Medium	1959	32.6%	38.1%	38.4%
High	720	12.0%	68.5%	66.8%

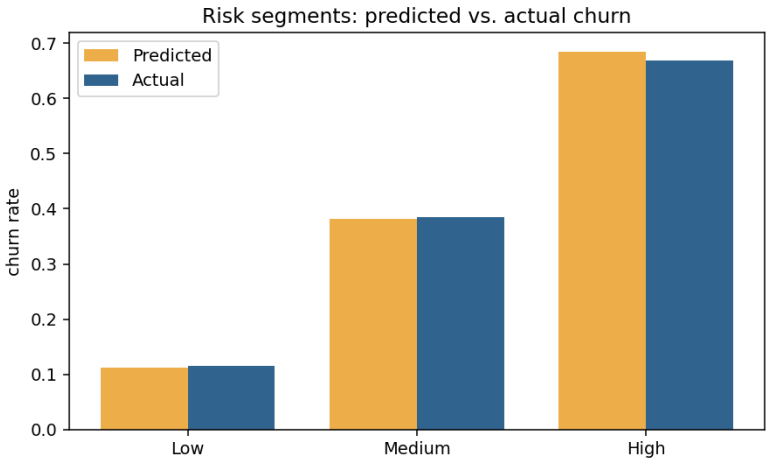


Figure 4 - Predicted vs. actual churn by segment

4. Business impact simulation (illustrative)

Assumptions (placeholders, replace with real numbers): $20 cost per retention offer, $400 value per retained customer, 30% of contacted churners retained.

Top % contacted	Customers	Churners reached	Expected saved	Net value ($)
5	300	223	66.9	20,760
10	600	417	125.1	38,040
20	1200	719	215.7	62,280

Top % contacted	Customers	Churners reached	Expected saved	Net value ($)
30	1800	972	291.6	80,640
40	2400	1156	346.8	90,720
50	3000	1303	390.9	96,360

Under these assumptions targeting by model score yields positive net value at every contact level; the marginal gain flattens beyond ~40% of customers, so the optimal campaign size depends on offer cost and capacity.

Appendix - Source code

src/deep_learning.py

```
"""TASK 5a - Deep learning: a multilayer neural network (MLP) on the same pipeline.

TensorFlow/PyTorch are not installed in this environment, so the network is built with
scikit-learn's MLPClassifier (dense layers, ReLU, Adam, L2, early stopping). The code is
structured so the estimator can be swapped for a Keras model wrapped with SciKeras.
"""
import json
import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from .config import FIGS, MODELS, RESULTS, SEED
from .features import make_pipeline
from .train_models import load_train

ARCHS = {"64": (64,), "128-64": (128, 64), "128-64-32": (128, 64, 32)}


def mlp(arch):
    return MLPClassifier(hidden_layer_sizes=arch, activation="relu", solver="adam", alpha=1e-3,
                         batch_size=256, learning_rate_init=1e-3, early_stopping=True,
                         validation_fraction=0.1, n_iter_no_change=10, max_iter=200, random_state=SEED)


def main():
    X, y = load_train()
    Xa, Xv, ya, yv = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
    val = {}
    for name, arch in ARCHS.items():
        p = make_pipeline(mlp(arch)).fit(Xa, ya)
        val[name] = round(float(roc_auc_score(yv, p.predict_proba(Xv)[:, 1])), 4)
        print("arch", name, val[name], flush=True)
    best = max(val, key=val.get)
    final = make_pipeline(mlp(ARCHS[best])).fit(X, y)       # refit on full train set
    net = final.named_steps["model"]
    joblib.dump(final, MODELS / "mlp_neural_net.joblib")

    fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
    ax[0].plot(net.loss_curve_, color="#d1495b"); ax[0].set_title("Training loss (log-loss)")
    ax[0].set_xlabel("epoch"); ax[0].set_ylabel("loss")
    ax[1].plot(net.validation_scores_, color="#00798c"); ax[1].set_title("Validation accuracy (early stopping)")
    ax[1].set_xlabel("epoch"); ax[1].set_ylabel("accuracy")
    plt.tight_layout(); plt.savefig(FIGS / "t5_mlp_training.png", dpi=140); plt.close()

    out = {"architectures_val_auc": val, "selected": best, "layers": list(ARCHS[best]),
           "activation": "relu", "optimizer": "adam", "l2_alpha": 1e-3, "batch_size": 256,
           "epochs_run": int(net.n_iter_), "best_val_accuracy": round(float(net.best_validation_score_), 4),
           "final_train_loss": round(float(net.loss_), 4),
           "n_parameters": int(sum(w.size for w in net.coefs_) + sum(b.size for b in net.intercepts_))}
    (RESULTS / "task5_deep_learning.json").write_text(json.dumps(out, indent=2))
    print(json.dumps(out, indent=2))


if __name__ == "__main__":
    main()
```

src/predictive_analytics.py

```
"""TASK 5b - Predictive analytics on the selected model.

Permutation feature importance, decile lift / cumulative gains, risk segmentation and an
illustrative retention-campaign ROI simulation. Writes reports/test_predictions.csv that
feeds the dashboard.
"""
import json
import joblib
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.inspection import permutation_importance
from .config import DATA_PROC, FIGS, MODELS, REPORTS, RESULTS, RISK_BINS, RISK_LABELS, SEED, TARGET

COST_PER_OFFER, VALUE_OF_SAVED_CUSTOMER, SAVE_RATE = 20.0, 400.0, 0.30   # illustrative assumptions


def main():
    meta = json.loads((MODELS / "metadata.json").read_text())
    pipe = joblib.load(MODELS / "best_model.joblib")
    test = pd.read_csv(DATA_PROC / "test.csv")
    X, y = test.drop(columns=[TARGET]), test[TARGET]
    p = pipe.predict_proba(X)[:, 1]
```

```
    pred = pd.DataFrame({"customer_id": test["customer_id"], "churn_probability": p.round(4),
                         "predicted_churn": (p >= meta["threshold"]).astype(int), "actual_churn": y})
    pred["risk_segment"] = pd.cut(pred["churn_probability"], RISK_BINS, labels=RISK_LABELS, right=False)
    pred.to_csv(REPORTS / "test_predictions.csv", index=False)

    # 1) permutation importance on the RAW input columns
    feats = [c for c in X.columns if c != "customer_id"]
    idx = X.sample(3000, random_state=SEED).index
    imp = permutation_importance(pipe, X.loc[idx, feats], y.loc[idx], scoring="roc_auc",
                                 n_repeats=5, random_state=SEED, n_jobs=1)
    imp_df = pd.DataFrame({"feature": feats, "importance": imp.importances_mean,
                           "std": imp.importances_std}).sort_values("importance", ascending=False)
    top = imp_df.head(12).iloc[::-1]
    plt.figure(figsize=(7, 5)); plt.barh(top["feature"], top["importance"], xerr=top["std"], color="#30638e")
    plt.xlabel("Drop in ROC-AUC when feature is shuffled"); plt.title("Permutation feature importance (top 12)")
    plt.tight_layout(); plt.savefig(FIGS / "t5_importance.png", dpi=140); plt.close()

    # 2) decile lift / gains
    pred["decile"] = pd.qcut(pred["churn_probability"].rank(method="first", ascending=False), 10, labels=range(1, 11))
    base = pred["actual_churn"].mean()
    dec = pred.groupby("decile", observed=True).agg(customers=("actual_churn", "size"), churners=("actual_churn", "sum"),
                                                    avg_prob=("churn_probability", "mean")).reset_index()
    dec["churn_rate"] = dec["churners"] / dec["customers"]
    dec["lift"] = dec["churn_rate"] / base
    dec["cum_gain"] = dec["churners"].cumsum() / dec["churners"].sum()
    fig, ax = plt.subplots(1, 2, figsize=(11, 4))
    ax[0].bar(dec["decile"].astype(int), dec["lift"], color="#00798c"); ax[0].axhline(1, color="k", ls="--", lw=.8)
    ax[0].set_title("Lift by risk decile (1 = highest risk)"); ax[0].set_xlabel("decile"); ax[0].set_ylabel("lift vs. average")
    ax[1].plot([0] + list(range(10, 101, 10)), [0] + list(dec["cum_gain"] * 100), marker="o", color="#d1495b", label="model")
    ax[1].plot([0, 100], [0, 100], "k--", lw=.8, label="random"); ax[1].legend()
    ax[1].set_title("Cumulative gains"); ax[1].set_xlabel("% customers contacted"); ax[1].set_ylabel("% churners captured")
    plt.tight_layout(); plt.savefig(FIGS / "t5_lift_gains.png", dpi=140); plt.close()

    # 3) risk segments
    seg = pred.groupby("risk_segment", observed=True).agg(customers=("actual_churn", "size"), avg_pred=("churn_probability",
        "mean"),
                                                          actual_rate=("actual_churn", "mean")).reset_index()
    seg["share"] = seg["customers"] / len(pred)
    plt.figure(figsize=(6.5, 4)); x = np.arange(len(seg)); w = .38
    plt.bar(x - w / 2, seg["avg_pred"], w, label="Predicted", color="#edae49")
    plt.bar(x + w / 2, seg["actual_rate"], w, label="Actual", color="#30638e")
    plt.xticks(x, seg["risk_segment"].astype(str)); plt.ylabel("churn rate"); plt.title("Risk segments: predicted vs. actual
        churn")
    plt.legend(); plt.tight_layout(); plt.savefig(FIGS / "t5_segments.png", dpi=140); plt.close()
    # 4) illustrative ROI of contacting the top-k% by risk
    order = pred.sort_values("churn_probability", ascending=False).reset_index(drop=True)
    roi = []
    for k in [5, 10, 20, 30, 40, 50]:
        top_k = order.head(int(len(order) * k / 100))
        saved = top_k["actual_churn"].sum() * SAVE_RATE
        net = saved * VALUE_OF_SAVED_CUSTOMER - len(top_k) * COST_PER_OFFER
        roi.append({"top_pct": k, "contacted": len(top_k), "churners_reached": int(top_k["actual_churn"].sum()),
                    "expected_saved": round(saved, 1), "net_value": round(net, 0)})
    out = {"importance": imp_df.head(12).round(4).to_dict("records"), "deciles": dec.round(4).to_dict("records"),
           "segments": seg.assign(risk_segment=seg["risk_segment"].astype(str)).round(4).to_dict("records"),
           "roi": roi, "assumptions": {"cost_per_offer": COST_PER_OFFER, "value_of_saved_customer": VALUE_OF_SAVED_CUSTOMER,
                                       "save_rate": SAVE_RATE}, "top_decile_lift": round(float(dec.loc[0, "lift"]), 2),
           "top_20pct_gain": round(float(dec.loc[1, "cum_gain"]), 3)}
    (RESULTS / "task5_analytics.json").write_text(json.dumps(out, indent=2))
    print(json.dumps({k: out[k] for k in ["segments", "roi", "top_decile_lift", "top_20pct_gain"]}, indent=1))
    print(imp_df.head(8).round(4).to_string())


if __name__ == "__main__":
    main()
```